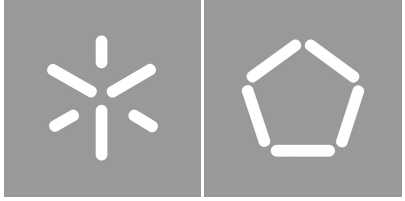
University of Minho
School of Engineering

Tiago Miguel Moreira Bacelar Pereira

Simulation of Hybrid Systems via Exact Real Arithmetic

december 2025



University of Minho
School of Engineering

Tiago Miguel Moreira Bacelar Pereira

Simulation of Hybrid Systems via Exact Real Arithmetic

Master's Dissertation in Informatics Engineering

Dissertation supervised by
Renato Neves

december 2025

Contents

1 Introduction **1**

1.1 Context and Motivation 1

1.2 Goals 3

1.3 Overview 4

1.4 Document Structure 4

2 State of the Art **6**

2.1 The Theory of Exact Real Arithmetic 6

2.1.1 Cauchy Sequences 6

2.1.2 Domain theory and Interval Arithmetic 7

2.1.3 Modulus of Convergence 10

2.1.4 Precision vs Accuracy 11

2.1.5 Summary 12

2.2 Implementations: from Theory to Practice 13

2.2.1 Technical Considerations 13

2.2.2 Minimal Features/Formal Definition 14

2.2.3 Package Survey 17

2.2.4 Comparison and Summary 19

2.3 The Fundamentals of Hybrid Systems 20

2.3.1 Formalizing Hybridness 20

2.3.2 The Hybrid Monad(s) 21

3 Implementation of a Hybrid Simulator **23**

3.1 System Architecture 23

3.2 Parsing a Hybrid Language 23

3.3 A Hybrid Interpreter 23

3.3.1 Iteration Limit . . . 23
3.3.2 Language Semantics: Formalization . . . 24
3.4 The Challenges of Undecidability . . . 24
3.4.1 Strict Comparison . . . 24
3.4.2 Boolean Expressions . . . 25
3.4.3 Lax Comparison . . . 26
3.4.4 Runtime Errors . . . 27
3.4.5 Hybrid Undecidability . . . 27
3.5 Differential Equation Solver . . . 28
3.6 Visualizer . . . 29
3.6.1 Discontinuity Tracking . . . 29

4 Results **31**

5 Future work **32**

List of Figures

1 Plot of the velocity over time in the cruise control program 2

2 Plot of x over time in the exponential program 3

3 Hasse diagram of the domain of the kleenians 8

4 Hasse diagram of the ordering domain 8

5 Core functionality of *CompReal* (abbreviated as CR) 15

6 A transparent implementation of the numerical sum of two *CompReals* (abbreviated as CR) 17

Chapter 1

Introduction

1.1 Context and Motivation

Hybrid systems are commonly defined as processes that possess a mix of both continuous evolution, usually modeled through differential equations, and discrete behaviors, such as conditionals, assignments and other instantaneous changes **?**. This model proves invaluable, specifically when simulating the interaction of physical, continuous processes with software-controlled systems, such as cruise controllers and pacemakers. The resulting programs are known as hybrid programs.

Taking the example of a cruise controller, we can write a simple program that checks the velocity of a vehicle every second and either accelerates or brakes, eventually hovering around the target velocity. The following program was implemented in Lince **?**, an imperative hybrid programming language and simulator:

```
1 v:=4;
2 while true do {
3     if v<=10 then v'=1 for 1; else v'=-1 for 1;
4 }
```

Listing 1.1: Example of a hybrid program

The program starts by assigning the value 4 to variable v. The statement inside the while loop will either accelerate or break, depending on the current velocity, by setting the derivative of v, v', to 1 or -1 during 1 second. The while loop indicates that this behavior repeats indefinitely. This appears to be a non-terminating program, because of the infinite while loop, but the evaluation at any specific time will only go through a finite number of iterations of the loop. A plot of the simulated velocity over time can be found in Figure 1.

Despite simple, this example illustrates the basic concepts of a hybrid program, namely the presence of both classical assignment, control structures, etc. as well as differential, continuous constructs and

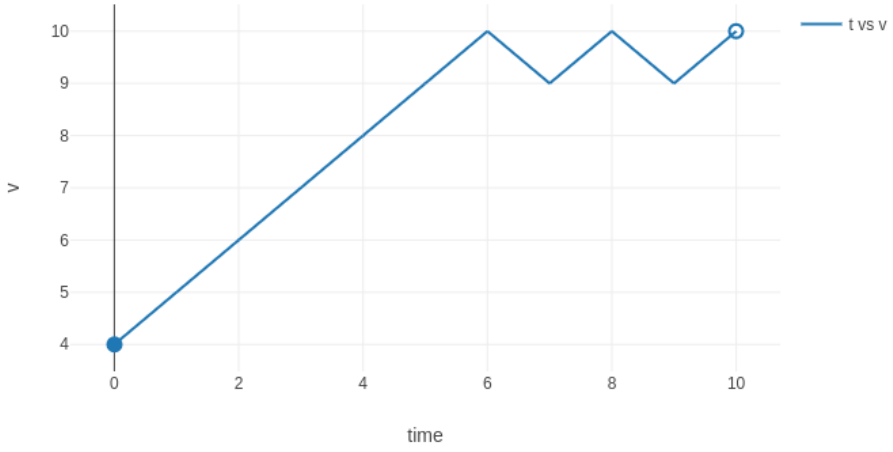


Figure 1: Plot of the velocity over time in the cruise control program

their relation to time.

Among the many difficulties of this paradigm, one of the greatest is the often overlooked matter of precision and error. When developing proper methods of simulation, the standard way to represent numbers is floating point arithmetic, which treats values as a mantissa and an exponent. This system can be, and sometimes is, used to represent numbers with an arbitrary precision, but is usually implemented with a finite precision by limiting the size of the mantissa and exponent. This is enough for most applications, but especially in chaotic systems, where errors compound, the precision limit can be easily exhausted, and any value outputted by the program will stray further and further from the actual value of the calculation. Lince itself uses standard floating point numbers with 64 bits of precision, known as double in most programming languages, which makes it prone to errors when the values get too close to zero. Take the following program as an example:

```
1 x := 1;
2 x' = -x for 80;
3 x' = x for 80;
```

Listing 1.2: Hybrid program showcasing insufficient precision

Figure 2 is a plot of the simulation of variable x over time. Variable x was expected to go from 1 to a very small number at time $t = 80$ following a negative exponential (e^{-t}), and then follow a positive exponential (e^{t}) afterwards and bounce back to 1 at time $t = 160$. But the Lince simulator shows the final value of x as 5.653719, which is clearly an error. This error grows even larger if we replace 80 with even larger values.

Using an arbitrary number of digits to represent values, together with ratio representations (numer-

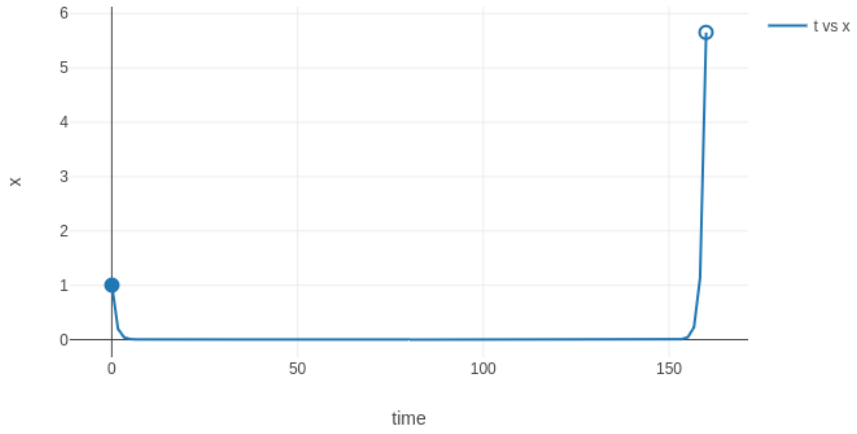


Figure 2: Plot of x over time in the exponential program

ator and denominator), allows for the exact representation of all rational numbers, but still leaves open the problem of irrational numbers, which are common when simulating physical systems (when using trigonometric functions, for instance, or with the previous exponential example). Unlike their rational counterparts, it would take an infinite number of digits to represent an irrational number using a regular digit representation. This problem therefore calls for alternative solutions.

1.2 Goals

The goal of the project is the simulation of hybrid systems through the use of exact real arithmetic. More specifically, the project consists of implementing a hybrid system simulator based on preexisting libraries for exact real arithmetic. Simulator performance is expected to be worse than simulators using floating point arithmetic such as the aforementioned tool Lince **?**, but should be perfectly faithful to the real system simulated.

The hybrid systems' specification language will be based on a while language with specific constructs to describe the behavior of physical processes. This choice is due to the simplicity and power of the language, which furthermore has a well-established operational semantics that can be used as a basis for implementation. The simulator will be implemented in the Haskell programming language due to its lazy paradigm and use of higher-order functions, which are particularly useful in the context of exact real arithmetic.

1.3 Overview

As a theoretic work, this thesis is mostly about compiling and systematizing already existing research from the fields of exact real arithmetic and hybrid systems. There is a wealth of research on exact real arithmetic, mostly on its more theoretical aspects (a good summary of which can be found in **?**). This thesis looks at multiple representation strategies from a more practical angle, how they are related, and their unique advantages and disadvantages. As for hybridness, since the topic is already well studied and systematized in the literature, it is presented here in a mostly non-transformative manner, and limited only to the more relevant concepts for the practical implementation of the simulator.

Original theoretical contributions only appear when the two areas intersect, namely: when performing comparisons of real numbers, which are needed for the hybrid simulator; and when numerically solving systems of differential equations using said real numbers. Both of these are made difficult by the specific limitations imposed by exact real arithmetic and have to be handled accordingly. The comparison problem is handled by partially evaluating the comparison up to some specified precision, raising an error if this precision is exceeded. As for the differential equations, despite the existence of multiple sophisticated algorithms for numerically solving ODEs, none of them fits our needs since they work with estimations and lack a strict error bound. Instead, the presented solution converts the equations into a system of polynomial equations, which have a stricter error bound for their Taylor series and do not require interval arithmetic **?**. Since no generic algorithm for the polynomial projection was found in the literature, one was developed by iterating the graph of values and derivatives of each sub-expression in the equations.

ref

On the practical side, this thesis is a comprehensive exposition of the inner workings and functionality of the developed simulator. Though some efforts were made to formalize the developed work, a lot more could be done in the direction of a solid formal foundation, with the ultimate goal being linking the operational and denotational semantics of the language and using the latter to prove its correction.

adjustar depois dependendo do que eu fizer

1.4 Document Structure

After this introduction, chapter 2 gathers the technical background relevant for this project. It first delves into an exploration of existing formalizations and techniques for representing exact real numbers (section 2.1). A study of several open-source libraries follows in section 2.2, where the underlying approaches are compared and critiqued. Then, section 2.3 lays out the theory behind hybrid systems and presents the underlying challenges of their simulation.

chapter 3 discusses the implementation of the simulator (architecture overview in section 3.1, followed by the parser in section 3.2, interpreter in section 3.3, solver in section 3.5 and visualizer in section 3.6), and exposes the challenges found along the way and their respective solutions, along with other notable implementation details.

In chapter 4 the performance of the simulator is discussed, and benchmarking results are presented.

Finally, chapter 5 talks about the general future direction of this research and lists some of the next steps towards it.

Throughout this thesis, the reader is assumed to be knowledgeable about partial orders, limits, derivatives, Taylor and Maclaurin series, and monads. Monads will be represented with a bold upper-case sans-serif letter (e.g. $\mathbf{M}$), and their respective functors are represented by the same letter in regular font weight (e.g. M). The set of natural numbers is represented with $\mathbb{N}$ and is understood to consist of all non-negative integers ($0 \in \mathbb{N}$)

Chapter 2

State of the Art

2.1 The Theory of Exact Real Arithmetic

There is a lot of established theory on exact real numbers, a good overview of which can be found in **?**. This section presents the basic concept behind a great deal of real number representations, namely infinite sequences of approximations, and explains the mathematical motivations behind it (subsection 2.1.1). Domain theory is discussed as both a motivation and formalization of the use of intervals as approximations in subsection 2.1.2, and the concept of modulus of convergence is presented in subsection 2.1.3. subsection 2.1.4 presents the formal definitions of precision and accuracy from numerical analysis and how they are used in the context of exact real arithmetic. Lastly, subsection 2.1.5 summarizes the whole section and lists some limitations common to all representation strategies.

2.1.1 Cauchy Sequences

There are multiple formal definitions of real numbers, but the most used definition for exact real number representation is based on Cauchy sequences. Concisely: the set of real numbers $\mathbb{R}$ is the smallest set that contains the rationals and is closed under limits of Cauchy sequences. So a simple and straightforward way to represent a real number is simply as a sequence of rationals. Of use to us, because the rational numbers are dense, this means that any real number can be approximated by a Cauchy sequence of rationals, or indeed any dense subset of the rationals, such as the dyadic numbers (numbers of the form $a/2^p$, with $a, p \in \mathbb{Z}$).

A Cauchy sequence is defined as a sequence whose terms get arbitrarily close to each other, essentially converging on a single number. More formally, $\{a_i\}_{i\in\mathbb{N}}$ is a Cauchy sequence if

$$\forall \epsilon > 0, \exists i : \forall j > i, |a_i - a_j| < \epsilon \tag{2.1}$$

This formalization, however, poses a problem: even if a sequence is known to be Cauchy, it is im-

possible to know which number it is converging into by looking only at a finite number of its terms. For example, the following sequence represents the number 2:

$$\begin{cases} a_i = 0 & \text{if } i < 1000000 \\ a_i = 2 & \text{if } i \geq 1000000 \end{cases} \tag{2.2}$$

In the worst-case scenario, the sequence may not even be Cauchy and not converge. A direct implementation of this concept, also known as the plain or naive Cauchy representation, is therefore insufficient, since it makes no guarantees about the represented real value at any step.

While insufficient *per se*, Cauchy sequences can still be used as a basis for more sophisticated representation methods. The next sections offer improvements to this basic idea.

2.1.2 Domain theory and Interval Arithmetic

Domain theory **?** deals with sets equipped with a complete partial order (cpo, here represented as $\sqsubseteq$), called domains, where this order measures the information content of each element of the domain. Each element of a domain can be thought of as a (partial) result of a computation, and one element is greater than another if it extends its information content in a consistent way. Like any partial order, cpos must be reflexive, transitive and antisymmetric. Additionally, every directed chain of the domain (that is, a chain of elements where each element is greater than the previous), must have a least upper bound in the domain. Informally, there is an element of the domain that summarizes all information of any non-contradicting chain of the domain. This property is called completeness.

For example, take the kleenians ($K_3 = \{T, F, U\}$), the 3 possible valuations of a proposition in Kleene's three-valued logic, first introduced in **?**. This domain is of special interest to this dissertation since it will form the basis for the evaluation of propositions in the simulator (subsection 3.4.2). The kleenians can be understood as a domain $(K_3, \sqsubseteq)$ where its elements are related as follows: $U \sqsubseteq F$ and $U \sqsubseteq T$ (Figure 3). This relation represents the fact that we may have no information about the value of a proposition (U), or have mutually exclusive information (the proposition is T or F).

Another example of a domain, also of interest later in the dissertation, is the ordering domain, which extends the 3 relative positions obtained from comparing two real numbers ($a < b$ - LT, $a = b$ - EQ - and $a > b$ - GT) into a domain, where each element of the domain represents a possible state of knowledge about the relative position of the two reals, as shown in Figure 4:

Domain theory is the basis of denotational semantics of programming languages, since it models the idea of computational information. A computation can be thought of as a continuous function, that is, a

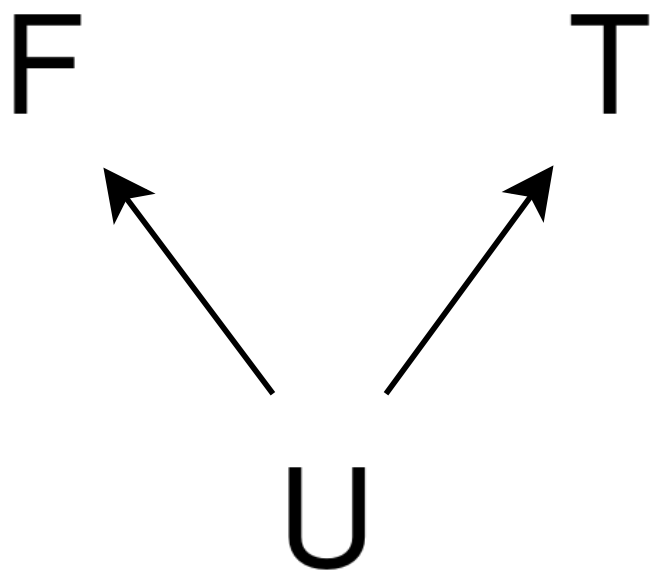


Figure 3: Hasse diagram of the domain of the kleenians

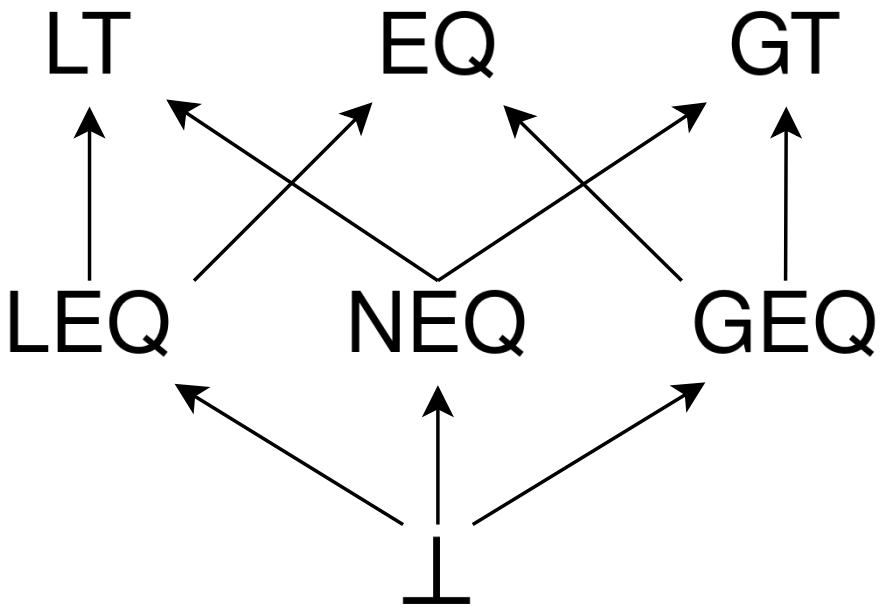


Figure 4: Hasse diagram of the ordering domain

monotonic function such that, for any subset X of the domain:

$$\bigsqcup_{x \in X} f(x) = f(\bigsqcup_{x \in X} x) \tag{2.3}$$

where $\bigsqcup$ denotes the least upper bound. Another recurring concept in domain theory is the bottom element $\bot$ of a domain D, which, when it exists, is defined as the least element of D ($\forall x \in D, \bot \sqsubseteq x$ - that is, $\bot$ is the only minimal element of D) and denotes no information or a never-ending computation. In the previous examples, both domains have a bottom element.

To represent real numbers, it is interesting to work specifically with the domain of closed real intervals, the reason for which will shortly become apparent. We define the domain $(I, \sqsubseteq)$ as:

$$[a, b] \sqsubseteq [c, d] \iff [a, b] \supseteq [c, d] \tag{2.4}$$

from which we can derive

$$\bigsqcup_{[a,b] \in X} [a, b] = \bigcap_{[a,b] \in X} [a, b] \tag{2.5}$$

This means that an interval has greater information content than another if it is contained in it. In this domain, a maximal element is an interval with zero length, that is, an interval of the form $[a, a]$, since the only interval contained in it is itself ($[a, a] \supseteq [b, c] \rightarrow [a, a] = [b, c]$, or equiavalently, $[a, a] \sqsubseteq [b, c] \rightarrow [a, a] = [b, c]$, which is the definition of a maximal element in $\sqsubseteq$). $[a, a]$, therefore, denotes exactly the real number a. Crucially, because computations correspond to continuous functions on the domain, any computation on a set of converging intervals will approach the result of the computation applied to the denoted real number.

This suggests how a possible implementation might be structured **?**: a real number should be represented by a sequence of nested rational intervals, so that each interval of the sequence collects all of the information of the previous ones. This way, we have solid theoretical ground to know that an operation to the sequence of intervals will approach the result of the operation of the denoted real number, and that each resulting interval is at least as accurate as the previous ones.

As a small aside, it is important to keep in mind that operating on an interval is, in general, harder than operating simply on its endpoints. While that is the case for monotonic operations on real numbers, such as addition and subtraction (e.g. the sum of two intervals is the interval whose endpoints are the sum of the terms' endpoints), in general it may be necessary to consider the whole interval (when calculating the sine of an interval, for instance). As a result, exact real representations through sequences of nested

intervals have a greater computational cost on operations than plain Cauchy sequence representations.

2.1.3 Modulus of Convergence

Although nested intervals allow the sequence to bound every approximation's error, note that it does not actually guarantee that any sequence converges on a single real number; it may simply converge on an interval. To ensure the former, we must introduce what is known as a modulus of convergence. The modulus of convergence is defined as follows: for a given Cauchy sequence $\{a_i\}_{i\in\mathbb{N}}$, the modulus of convergence is a function $f : \mathbb{N} \to \mathbb{N}$ that satisfies the following relation:

$$i \geqslant j > f(n) \implies |a_i - a_j| \leq 2^{-n} \tag{2.6}$$

Informally, for a certain n, the modulus of convergence gives us a position on the sequence such that all future terms differ less than 2^{-n} from each other, therefore implying that all of these terms have an accuracy of at least n. If such a function exists, it is possible to prove that the sequence follows Equation 2.1 and therefore converges.

To avoid treating the sequence and its modulus separately, an exact real number representation may enforce that its sequences follow a fixed modulus of convergence $f(n) = n$, to which any arbitrary sequence can be converted by composing with its modulus of convergence: $[c, d]_i = [a, b]_{f(i)}$.

In practice, many representations don't provide a modulus of convergence, since calculating and enforcing it comes with added computational costs, and instead settle for nested intervals. When they do, the only way to evaluate the represented real number up to a certain accuracy is to advance through the sequence until the wanted accuracy is met.

Lastly, note that a Cauchy sequence $\{a_i\}_{i\in\mathbb{N}}$ equipped with a modulus of convergence $f : \mathbb{N} \to \mathbb{N}$ can be converted into a valid sequence of nested intervals $\{[b, c]_n\}_{i\in\mathbb{N}}$, since the modulated terms of the sequence effectively form an interval of length 2^{-n}:

$[b, c]_n = [a_{f(n)} - 2^{-n-1}, a_{f(n)} + 2^{-n-1}]$

This makes nested intervals a sort of intermediate step between the basic Cauchy sequences with no modulus, the least restrictive representation (so nonrestrictive, in fact, that it is insufficient for practical use), and sequences with a modulus, which have the most restrictions. For example, to represent the number 2, a plain Cauchy sequence has literally no restrictions on any finite number of its elements (see the example at the end of subsection 2.1.1); a sequence of nested intervals forces all its intervals to contain the number 2 (but does not restrict the length of the intervals in any way); and a modulus of convergence sets a maximum interval length that gets cut in half for each new interval.

2.1.4 Precision vs Accuracy

Since these two terms are used interchangeably in day-to-day language, it may be helpful to revisit their formal definition in numerical analysis and how they apply in exact real arithmetic in particular.

Accuracy refers to how close the result of a computation or approximation is to the mathematical true value being computed, measured by the absolute or relative error of the approximation. In this thesis, accuracy is denoted by n [1] and is defined logarithmically in terms of the absolute error ϵ:

$$n = \lfloor -log_2(\epsilon) - 1 \rfloor$$

This quantity is, not incidentally, the input of the modulus of convergence of a Cauchy sequence. If the accuracy n of a certain result or approximation $\hat{x}$ is known, we can construct an interval of length 2^{-n} of all possible values for the computation's true value x:

$$\hat{x} - 2^{-n-1} \leq x \leq \hat{x} + 2^{-n-1}$$

For instance, take $50/16 = 3.125$ as an approximation of pi. This approximation has an error of $\epsilon = |\pi - 50/16| = 0.016592...$, from which we derive its accuracy of $n = \lfloor -log_2(0.016592...) - 1 \rfloor = 4$. This means that in a Cauchy sequence converging to pi that follows a constant modulus of convergence ($f(n) = n$), this approximation could be any of the first five terms. Of course, this example is, in a sense, "backwards", since we used the actual value of pi to calculate the error (and then accuracy) of our approximation. In reality, the reason why we generate approximations is to approach an unknown true value, and so the error has to be inferred from the nature of the operations performed to obtain the approximation. This process of error inference is an important part of numerical analysis, and extremely relevant to us here as well: we want to generate error bounds as tight as possible for the amount of computational resources expended, so as to maximize performance.

Precision, on the other hand, is the resolution of the result's representation, usually given as the number of decimal or binary digits. More formally, *Accuracy and Stability of Numerical Algorithms* **?** defines precision as "the accuracy with which the basic arithmetic operations $+$, $-$, $*$, $/$ are performed". As an example, take dyadic numbers: for a fixed p and any integer a, dyads of the form $a/2^p$ may be said to have a precision of p, since operations may cause errors of at most $1/2^{p+1}$ - exactly half of the "hole" between consecutive dyadic numbers[2].

This example also illustrates how the precision of dyadic numbers (and, indeed, any other arithmetic)

[1] As implied by the choice of symbol, n is a natural number ($n \in \mathbb{N}$). Although it wouldn't be nonsensical to consider negative accuracies as well (that is, absolute errors greater than 0.5), in practice, sequence of approximation-type representations are usually limited to natural accuracies for ease of implementation

[2] This is equivalent to how precision is usually discussed in computer science (in the context of floating-point numbers), but considering the alternative definition of accuracy using relative error instead of absolute error. Floating-point is essentially an attempt to minimize the relative error caused by the "holes" between consecutive floating-point numbers, while dyadic numbers attempt to minimize the absolute error

interacts with the accuracy of computations: while some computations (like addition and subtraction) can be performed with perfect accuracy (e.g. $1/2^0 + 2/2^0 = 3/2^0$), in the worst-case scenario (where the true value of a result is right in the middle of a "hole"; for example, $1/2^0 \div 2/2^0$), accuracy is limited by the precision of the approximation ($n \leq p$), independently of the result.

While precision is important when discussing the development and inner workings of an algorithm or calculation, end users are usually more interested in a result's accuracy, or, equivalently, on an error bound for the result. Consider the aforementioned example of $50/16 = 3.125$ as an approximation for pi. If it is interpreted as the dyadic number $50/2^4$, then this approximation has a precision of $p = 4$. However, notice that $51/2^4$, $-42/2^4$, or indeed *any* dyadic number with the same denominator also has a precision of 4, even though they aren't remotely close to pi. Even though $50/2^4$ is the *best* approximation with precision 4, it is far from the only one. That being said, it is true that *because* it is the best approximation with precision 4, $50/2^4$ will necessarily have an accuracy of (at least) 4.

It is worth mentioning that, in the context of arbitrary-precision (and arbitrary-accuracy) arithmetic, it makes no sense to talk about the precision or accuracy of a number, since each number can be approximated to arbitrarily high accuracy. Precision and accuracy are only relevant when discussing the number's approximations.

2.1.5 Summary

To summarize, all presented methods of exact real representation rely on sequences of increasingly accurate approximations. This concept is based on Cauchy sequences, which by themselves are insufficient since they don't make accuracy guarantees. Two refinements are possible: using sequences of shrinking intervals to keep track of the accuracy of the approximations at each step (nested intervals representation); and keeping a modulus of convergence, or following a fixed modulus, to make guarantees about both the accuracy and the speed of the convergence.

There are a few theoretical quirks to any exact real representation. Perhaps most surprisingly, not all real numbers can be computationally described. This appears to contradict what has been said so far - that any real number is described by a rational Cauchy sequence. However, infinite sequences cannot be stored directly in a computation device with a finite memory. Instead, to represent a sequence, a procedure or function that generates said sequence is used, which means many sequences (and by extension many real numbers) are impossible to represent. This can be formally proved independently of the concept of real numbers as sequences: since there are countably many programs (seeing as each program is a finite, though arbitrarily long, sequence of a finite number of symbols), they can only represent countably many

real numbers, no matter the chosen representation strategy. And a Cantor diagonal argument can be used to prove that there are, therefore, real numbers that cannot be represented. The set of representable real numbers is referred to in the literature as the computable real numbers.

This fact isn´t really a limitation; after all, if a number can´t be produced as the result of any (finitely describable) computation, then we will never come across it, and so not being able to represent it isn´t a big concern. What is a limitation, though, is the frustrating fact that the comparison of two computable real numbers is undecidable in the general case, since it may involve comparing infinitely many terms of a sequence when the two compared numbers are equal. This is also true when testing for (in)equality. For this reason, practical implementations of the presented ideas offer workarounds or alternatives to comparison, such as performing comparison up to some specified accuracy.

2.2 Implementations: from Theory to Practice

This chapter is dedicated to studying existing implementations of exact real arithmetic in the Haskell programming language. We will analyze each package's chosen representation strategy from a theoretical perspective, listing their features and respective pros and cons. chapter 4 presents an empirical analysis of the packages' performance.

2.2.1 Technical Considerations

Before starting, a couple of points are worth considering. Haskell, as a high-level functional programming language, provides an arbitrary precision *Integer* data type that greatly simplifies number representation. Although the theory discussed thus far assumes rationals as approximations for real numbers, all analyzed Haskell packages make use of dyadic numbers: numbers of the form $a/2^p$. What is useful about them is they can be represented as two *Integers* (or an *Integer* and an *Int*, the fixed precision[3] counterpart to *Integer*) and have simple sum and product operations, which benefit from bitshift optimizations of powers of 2 multiplication. The main disadvantage of dyadic numbers is that the representation of all other rational numbers also becomes an infinite sequence (in the same way that using a base 10 digit system makes the representation of $1/3$ an infinite sequence: 0.33333...). Haskell also provides a *Rational* datatype, which is stored as two *Integers* (numerator and denominator), which allows users to exactly represent all rational numbers.

[3] Although the exact size is implementation-dependent, with most implementations using 64 bits, an *Int* is guaranteed to have at least 30 bits. This can make it dangerous when used as the exponent of extremely small dyadic numbers

Another concept to have in mind is memoization, also known as caching. Memoization is relevant when the same intermediate calculation is used more than once to produce some desired result. In these circumstances, memoizing the intermediate result avoids pointlessly repeating calculations, therefore increasing performance. In Haskell, some degree of memoization can be done automatically by taking advantage of thunks - a single lazy unit of computation. Essentially, when assigning the result of a computation to a variable, the computation isn't performed until the value is used. If the value is used multiple times, the computation is only run the first time. This allows carefully structured programs to memoize results by storing them in variables or data structures. This is useful in intermediate calculations, since the variables used in them are automatically discarded when no longer needed, avoiding unnecessary memory consumption. That said, all memoization is fundamentally a trade-off of time costs for memory costs, and all of the techniques presented below increase memory use.

In the context of exact real arithmetic, there are two levels of memoization: when calculating the same sequence twice, or when calculating the terms of a sequence out of order. The first level is simple to implement: by storing each term of the sequence in a data structure (a list, for instance), they are cached for future use. The downside is that producing an approximation will require accessing the data structure, which in the case of lists comes with a linear cost.

The second level of optimization is more nuanced. It takes advantage of the observation that the n-th approximation of a real number is at least as accurate as any of the previous ones, and therefore, it is a valid result for all previous terms of the sequence. This is relevant when the sequence in question is the result of a long series of operations, and therefore each term has a high computational cost. Our observation allows us to return the already calculated n-th term when calculating a lesser term would require long computations, thus increasing performance. Note, however, that this makes the elements of the sequence dependent on whether later terms have already been calculated, which is an impure effect. This effect could in principle be modeled as a monad, but that would make using this hypothetical monadic implementation pretty cumbersome. The presented packages that perform this optimization opt for the use of unsafe Haskell functions to produce this impure effect.

2.2.2 Minimal Features/Formal Definition

better title

Each of the surveyed packages implements the concepts above (approximations, computable real numbers, etc.) as their own datatypes. However, for the purposes of testing, benchmarking and later developing the hybrid simulator, it is useful to group them into the same Haskell type class, which was named *CompReal*. This type class abstractly represents the concept of a computable real number, and

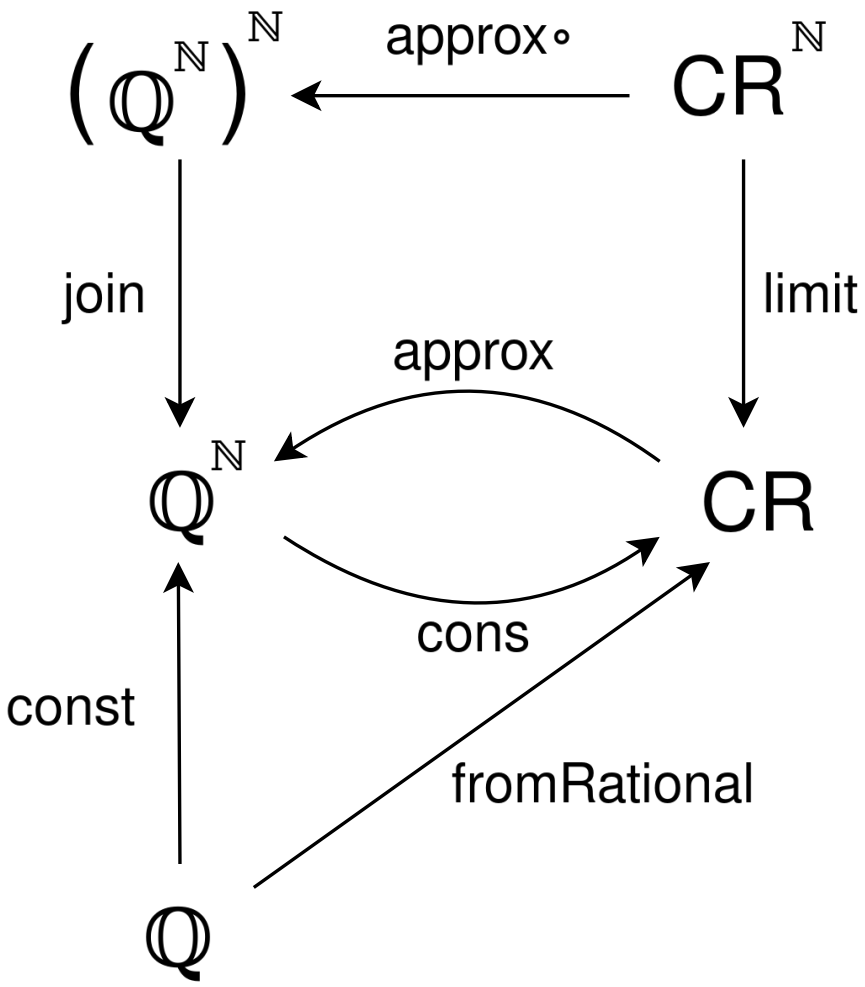


Figure 5: Core functionality of *CompReal* (abbreviated as CR)

specifies the following list of functionalities (most of which are pictured in Figure 5):

Since our theory is based on sequences of approximations, our computable reals should support being approximated up to a certain accuracy (as a *Rational*, to generalize any particular approximation datatype the libraries might have). In Figure 5 this functionality is provided by function *approx*. For example, take the following nested intervals representation of pi, $\{a_i\}_{i\in\mathbb{N}}$:

$a_0 = [3, 4]$ $a_1 = [3.1, 3.2]$ $a_2 = [3.14, 3.15]$ $a_3 = [3.141, 3.142]$ $a_4 = [3.1415, 3.1416]$ ...

Evaluating $approx\ a\ 5$ should result in an approximation with an accuracy of at least 5 (which means we want an approximation of error $\epsilon < 2^{-6} = 0.015625$ or, equivalently, an interval of width $2^{-5} = 0.03125$), so a_0 and a_1 must be skipped, since they are not tight enough for the required accuracy. Since a_2 has a width of 0.01, its centerpoint 3.145 is guaranteed to have an error of at most 0.005. Therefore, 3.145 is a valid approximation. Taking the centerpoint of any of the tighter intervals would also give a valid approximation, but in general generating tighter intervals requires higher computational costs, so it

is usually desirable to use the earliest possible interval to generate the approximation.

Computable reals should also support the reverse: a constructor that takes an approximation function or a list of approximations (*Rationals*), where the first term has an accuracy $n \geq 0$, the second term has $n \geq 1$, etc, and produces the real value they converge towards. This is pictured in Figure 5 as function *cons*. For instance, if we feed *cons* with the following geometric series, where $a = r = \frac{1}{2}$[4]:

$S_0 = \frac{1}{2}$ $S_1 = \frac{1}{2} + \frac{1}{4} = \frac{3}{4}$ $S_2 = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} = \frac{7}{8}$ ...

The resulting computable real $cons\ S$ will denote the number 1. Notice that $cons$ assumes a constant module of convergence, which in this particular case is satisfied, since the error of each approximation is $R_n = |S_\infty - S_n| = \frac{a}{1-r} - a\frac{1-r^{n+1}}{1-r} = |a\frac{r^{n+1}}{1-r}| = 2^{-n-1}$. For other geometric series, this condition might not hold, so some terms might have to be filtered out to ensure the filtered sequence follows the modulus. In any case, using $cons$ always requires some known bound to the error of each approximation, either to ensure they follows the modulus or to generate a sequence that does.

Obviously, computable reals must also support numerical operations. In addition to the basic arithmetic operations, real numbers are closed under trigonometric functions, exponentiation (with a positive base), and logarithms (on positive values). It must also be possible to promote any rational value to a computable real (this is function $fromRational$ in Figure 5, which can be obtained solely from the constructor as $cons \circ const$. Likewise, other operations such as addition, multiplication, etc can also be obtained from $approx$ and $cons$ - Figure 6 is a diagram illustrating such a construction for addition). These restrictions are specified by Haskell's *Floating* type class, which *CompReal* should extend.

Another natural operation to support is limits. Unlike the constructor, which simply constructs a computable real from a sequence of approximations (which is, as explained before, our theoretical basis for computable reals), a limit takes in a sequence of converging computable reals and produces a single computable real. This function is obtainable using *approx* and *cons* with the help of an auxiliary function $join : (\mathbb{Q}^{\mathbb{N}})^{\mathbb{N}} \to \mathbb{Q}^{\mathbb{N}}$, which chooses a sequence of approximations with a constant modulus from the inputted sequence of sequences.

Finally, real numbers should support comparison. As stated before, comparison is undecidable on the computable reals, so a straight implementation of haskell's *Ord* class is impossible. Instead, a new class *CompOrd* was created to capture the idea of comparing increasingly accurate approximations, in the domain theory sense. Its comparison function takes two values to compare and an accuracy n and returns an element of the *Ordering* domain, of which the top elements are *EQ*, *LT* and *GT*. This function can be written agnostically using the approximation functions provided by the *CompReal* class.

[4] The usual definition of a geometric series starts at index 1, since index 0 corresponds to adding zero terms of the series, which is always 0. This example uses the alternative definition $a_n = ar^n$ that starts at index 0, leading to the slightly different summation formula $S_n = a(\sum_{k=0}^{n} r^k) = a\frac{1-r^{n+1}}{1-r}$

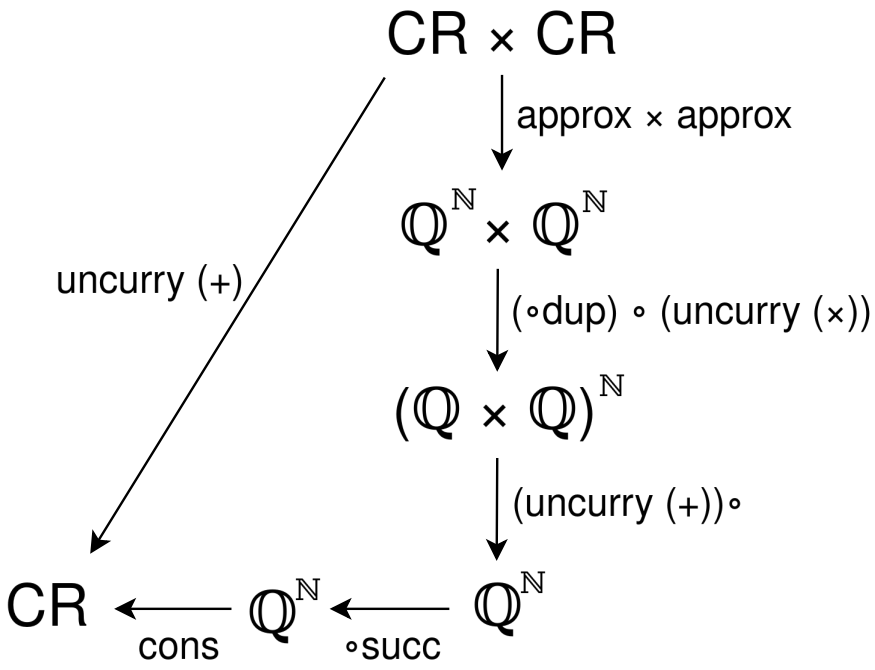


Figure 6: A transparent implementation of the numerical sum of two *CompReals* (abbreviated as CR)

An important point to make is that *CompReal* and $\mathbb{Q}^{\mathbb{N}}$ are **not** isomorphic; they might be, depending on the specific *CompReal*, but are not required to. Even though *approx* and *cons* are maps between the two types, their composition is not required to preserve the internal structure of the *CompReal* ($cons \circ approx \neq id$, or in other words, *approx* and *cons* are not each other's inverse), despite the resulting *CompReal* denoting the same real number. This change in internal structure can have serious impacts on performance, which is why these operations should be used judiciously. For this exact reason, even though *limit*, *fromRational* and all numeric operations could be implemented transparently, solely from manipulating sequences of approximations obtained with $approx$ and translating back to *CompReal* using *cons*, doing so would be terrible for performance. Instead, each instance of *CompReal* implements these functionalities independently, optimized to the specific internal structure of their representation, using its library's code and functionalities whenever possible and new code when not.

2.2.3 Package Survey

CDAR

CDAR **?** is based on centered dyadic approximations, described in **?**, and inspired by *iRAAM* **?**, an exact real arithmetic c++ library. Each approximation is stored as a center point and an error bound, both dyadic numbers with the same exponent. This is equivalent to storing the two endpoints of an interval,

as described in the paper, but is slightly more compact in terms of storage and has performance benefits when working with the error bound of the approximation. The downside is that numeric operations are harder to implement, since the image of an interval is not in general centered on the image of its center point (when performing multiplications, for example).

A real number is represented as a list of these approximations, taking advantage of Haskell's laziness. This way, it possesses the first degree of memoization discussed: in-order memoization. Operations using two real numbers are implemented as a zip of the two lists. Since the sequence of approximations doesn't follow a modulus of convergence, requesting an approximation with some given accuracy requires explicitly iterating the list until a suitable approximation is found.

aern2-real

aern2-real **?** is very similar to *CDAR*, also making use of centered dyadic approximations and lists. However, it implements a lot more utilities and operations, of note: a comparison function between two reals, returning an infinite list of kleenians (the three-valued logic equivalent of booleans: true, false or undecided); parallel branching capabilities receiving a kleenian; handling of partial functions and errors (using another package *collect-errors* **?** of the same author); and the calculation of limits as a built-in operation. Though most of these features aren't of use to our particular use case, an efficient implementation of limits is of the utmost importance to the hybrid simulator, since solving differential equations requires a lot of limits.

ERA

ERA **?** is the oldest and simplest of the surveyed packages. It represents real numbers as a Cauchy sequence of dyadic numbers with a fixed modulus of convergence, stored as a function *Int* $\rightarrow$ *Integer*, where each *Integer* is the numerator of a dyadic number. Formally, the real number r is represented by a function f if:

$$\forall j \in \mathbb{N}, |r - f(j)/2^j| < 2^j \tag{2.7}$$

check

Sadly, this implementation doesn't have many utilities to work with its real numbers other than the basic arithmetic operations. It also does not memoize any of its results, making it the slowest of the surveyed packages (detailed benchmarking results are examined in chapter 4).

exact-real

exact-real **?** uses the same approach as *ERA* with additional functionality, most notably, out-of-order memoization, implemented as a mutable variable for each real containing the most accurate dyadic approximation of the number calculated so far. It also includes the wanted accuracy of a real number in its type signature, which slightly complicates its syntax (and makes it awkward to work with in an arbitrary-accuracy context, requiring existential types and the like) but allows a straight implementation of Haskell's comparable and number classes since the information about the accuracy is present at the type level, which is neat.

ireal

ireal **?** expands on the ideas presented so far by merging the concepts of nested intervals and modulus of convergence into a single sequence of nested open intervals with dyadic endpoints with a fixed modulus of convergence, stored as a function $\mathit{Int} \to (\mathit{Integer}, \mathit{Integer})$, which is very close to the formalization of real numbers in domain theory (presented in subsection 2.1.2). This is important for *ireal*, since the creator wanted to ensure a solid theoretical ground for their implementation. Their paper explaining the implementation and showing its correctness can be found along with the source code.

Formally, an *ireal* value represented by a function $< f, g >$ contains a real number r if

$\forall j \in \mathbb{N}, f(j)/2^j < r < g(j)/2^j$

This representation is redundant when representing a single real number, since the condition imposed by the modulus already creates an interval of possible convergence points. However, this allows *ireal* to directly represent intervals with real endpoints. In this setup, a single real number is simply a sequence of thin intervals, that is, the j-th interval's endpoints are exactly $2/2^j$ apart.

Like *exact-real*, *ireal* also implements out-of-order memoization by making use of mutable variables, though better encapsulated. By its own admission, this is the least elegant part of the package, but essential for the performance of large computations.

2.2.4 Comparison and Summary

In a sense, the details of the individual implementations are irrelevant, insofar as all of them support (or can be extended to support) all of the same base features, specified in section 2.2.2. However, these details are still significant for performance.

To summarize how they impact performance:

Package	Strategy	Data Structure	Approximation	Memoization
CDAR	Nested Intervals	List	Centered dyad	Level 1
aern2-real	Nested Intervals	List	Centered dyad	Level 1
ERA	Modulus of Convergence	Function	Dyad	None
exact-real	Modulus of Convergence	Function	Dyad	Level 2
ireal	Modulus of Convergence	Function	Dyad Interval	Level 2

Table 1: Package comparison

- Strategy impacts how quickly the approximations converge. Modulus of convergence strategies are fixed and have little to no possibility of optimization, but have a guaranteed convergence speed. On the other hand, nested intervals can have any convergence, which means a lot more care has to be put into optimization
- Using lists as a data structure has the advantage of automatically granting level 1 memoization (which comes with performance benefits but also memory costs), but the disadvantage of forcing an explicit iteration to access later elements in the list. Functions can support level 2 memoization, but by default have none.
- The choice of approximation affects the way integer and rational values are represented, which may reduce the number of approximations needed. However, more complicated approximations have a greater overhead. All of the analyzed representations use dyadic numbers for their simplicity
- Memoization is absolutely crucial for performance, especially when real values are used multiple times for calculations. Level 1 stores the value of each approximation, causing more intensive memory use. Level 2 only stores the most accurate approximation calculated so far, which is only advantageous when the approximations are used out of order.

2.3 The Fundamentals of Hybrid Systems

2.3.1 Formalizing Hybridness

The current standard formalism for hybrid systems is hybrid automata. In essence, these are automata where states are labeled with differential equations (continuous behavior), and edges, marked with guards and variable assignments, determine the transition between the states (discrete behavior). While it is a

useful model, we don't actually need to consider automata to reason abstractly about hybridness. Instead, we will consider the basic features of hybrid systems in a functional lens and follow them to a monadic description of hybridness - the hybrid monad(s) - which will then be used to implement the concepts in practice.

The main characteristic of hybrid programs is their relation to time, namely, how the state of the program evolves through it. It is important to distinguish the simulated time from the real time, though: some hybrid programs can simulate hundreds of years instantly, while others can take hours to simulate a single second. When talking about time from now on, unless otherwise stated, it refers to simulated time, as opposed to execution time, performance, etc.

A hybrid program can be modeled as a function that, given a state, returns the evolution of the state through time, also known as a trajectory. For instance, given a state of n real numbers, a hybrid program would be a function of type $\mathbb{R}^n \to (\mathbb{R}^n)^{\mathbb{R}_{\geq 0}}$. Programs usually don't define the trajectory of the state indefinitely though, only up to some positive duration (d). This allows programs to be composed: the composition of two programs is a new program whose trajectory follows the first program's trajectory to its end and then feeds the resulting state into the second program and follows its trajectory.

2.3.2 The Hybrid Monad(s)

The features of hybridness presented in the previous section can be used to form a monad, as is described at length in **?**. In summary, the hybrid monad describes the effect of a continuous evolution during some time d, as expected. This monad's unit function, for some state x, generates an instantaneous "evolution" taking zero time and ending in x; and the multiplication function collapses an "evolution of evolutions" by evaluating them at their ends and adding their durations.

Modeling hybridness as a monadic effect greatly facilitates both reasoning about and implementing simulations of hybrid systems in practice: any instantaneous change in state, in the form of some traditional computation, can be converted into a hybrid program through the hybrid monad's unit function, and continuous evolutions are represented in general as a trajectory. This way, the hybrid monad represents the mix we were looking for of both discrete and continuous changes.

This general description of hybridness allows for any kind of state. However, as pointed out by the creators of Lince - an imperative hybrid programming language - in **?**, there are advantages in imposing additional constraints on the allowed state type. Due to its imperative nature, Lince only works with states of n real variables ($\mathbb{R}^n$), using differential equations as trajectory generators. As an advantage, instead of using the general hybrid monad **H** described in **?**, Lince formalizes its syntax with a parametrized monad

$\mathbf{H}_S$, where S is the state of the program (which in Lince's case happens to be $\mathbb{R}^n$ for some $n \in \mathbb{N}$).

TODO: aqui eu queria meter a definicao formal de $\mathbf{H}_S$ e mostrar que cumpre as leis, mas tenho algumas duvidas sobre a notacao do paper e sobre a minha implementação, e queria esclarecer isso primeiro quando reunirmos

Chapter 3

Implementation of a Hybrid Simulator

TODO: reescrever este paragrafo This chapter presents the implementation of this project's hybrid language, named Jaguar. It is organized into sections corresponding to each component described before: parser, interpreter, differential equation solver and visualizer. When relevant, formalizations are used to better detail the component being analyzed.

3.1 System Architecture

TODO: passar coisas ditas em **??** para aqui.

3.2 Parsing a Hybrid Language

To parse our hybrid language, the Happy parser generator was chosen. Happy takes an annotated BNF grammar specification and produces a Haskell module containing an LR parser for the grammar. Happy is the parsing library that GHC, the Haskell compiler, uses to parse its own syntax, and its pretty comprehensive functionalities proved to be more than enough for the project's needs. ref

The full specification of the language's syntax, in BNF notation and its precedence rules, is in the appendice/user guide . link

3.3 A Hybrid Interpreter

TODO: detalhes de implementacao (Hybrid, RunResult, etc)

3.3.1 Iteration Limit

TODO: zeno points

3.3.2 Language Semantics: Formalization

3.4 The Challenges of Undecidability

Previously, we saw features ubiquitous among many hybrid simulators. However, dealing with computable real numbers results in some unique challenges, mainly when it comes to comparison. In this section, these challenges and their respective solutions are presented and used as motivation for the formalization of Jaguar's semantics in subsection 3.3.2.

3.4.1 Strict Comparison

Take as an example the following program, which is meant to calculate the base 2 logarithm of a small number $x < 1$, rounded down ($\lfloor log_2 x \rfloor$):

```
1 x := 0.0001;  //can be any number less than 1
2 y := 1;
3 ans := 0;
4 while x < y do { ans := ans - 1; y := y / 2 }
```

Listing 3.1: Program to calculate the base 2 logarithm of x

This algorithm is mathematically correct: it always takes a finite number of steps for any input x, and always reaches the correct answer. However, notice what happens if we try to run the program with the input $x = 0.5$: the second iteration of the while loop will try to make the comparison $0.5 < 0.5$. Mathematically, it should evaluate to false, but as explained in section 2.1, comparing two equal computable real numbers will never terminate. As a result, our program will halt, even though it is mathematically sound!

The fundamental limitation here is the undecidability of comparison in the computable reals: whenever our hybrid simulator tries to fully evaluate a comparison, it may halt with no further feedback to the user. From a design standpoint, this places on the user the responsibility of ensuring that no comparison ever compares two equal numbers. Which, while in some cases is simply impractical, in others it is downright impossible.

Ideally, such comparisons would produce an error instead of halting. We can do that by evaluating the comparison only up to a certain precision. Formally, when evaluating $r < s$ to precision n, we first approximate r and s to intervals of rational numbers $a \leq r \leq b$ and $c \leq s \leq d$ such that $b - a, d - c \leq 2^{-n}$. If $b < c$, then we can be sure that $r < s$ and the original comparison is true. If

$d \leq a$, then we are sure $r \geq s$ and the comparison is false. In all other situations, we can't resolve the comparison; the given precision isn't enough. All that can be inferred is that $|r - s| \leq 2^{-n+1}$. In these cases an error is raised, giving valuable feedback to the user.

Analogous reasoning can be used for $>$, $\leq$ and $\geq$. However, operators for equality and inequality ($=$ and $\neq$) aren't included. The reason for this is simple: two equal numbers can't be guaranteed to be found equal, since this would require both numbers to be approximated to an interval of zero length (and while this isn't impossible - remember the restriction on intervals is $a \leq r \leq b$ such that $b - a \leq 2^{-n}$, so zero length is technically possible - it isn't guaranteed either, and depends entirely on the particular representation details of the two numbers). As a result, these hypothetical operators would always yield the same output: false in the case of $=$ and true in the case of $\neq$; or be unable to produce any output at all and raise an error.

Of course, if the comparison precision is too low, close comparisons ($|r - s| \leq 2^{-n+1}$ at most, depending on the approximations) can also result in an error; but these false positives are inevitable if we wish to prevent halting. It is up to the user to interpret the error, figure out the cause and decide how to solve it; either by raising the comparison precision, or by rewriting the program to avoid the faulty comparison, or even re-enabling full comparisons (by "raising the comparison precision to infinity") if they are confident the program will never halt.

The comparison precision n could be specified for each comparison individually, but to avoid cluttering the syntax it was decided instead that all comparisons in a program would have the same precision, which can be supplied as an external configuration when running the program.

3.4.2 Boolean Expressions

Since comparisons may not produce a result, boolean expressions using those comparisons have to deal with three possible outputs for a comparison: true, false and inconclusive. In the previous sections, it was implied that any inconclusive comparison would immediately raise an error, but this doesn't have to be the case. Take, as an example, the following program:

```
1 x := 1;
2 y := 1;
3 while x > 0.125 || y < 30 do { x := x / 2; y := y * 2; }
```

Listing 3.2: Hybrid program showcasing the usefulness of three-valued logic

Although the comparison $x > 0.125$ is indeterminate at the beginning of the fourth iteration, the whole expression can be inferred to be true: since $y < 32$ is true, regardless of the value of the first term,

the disjunction is true. In a sense, we can treat indeterminate values as simply another possible output from a comparison, and then work out the possible results of the disjunction. If both true and false results are possible, then the whole disjunction is indeterminate.

Essentially, what is being described is a three-valued logic as a way to capture the notion of indeterminacy. The topic is discussed in length in TODO. Interestingly, this notion can also be conceptualized from a domain theory lens as the flat domain of booleans, containing three values: true, false and bottom. Such a formalization is useful because it allows us to extend what was said before about computable real numbers (subsection 2.1.2) to the semantics of Jaguar: evaluating boolean expressions will approach the actual result of the denoted condition, and expressions evaluated with greater comparison precision are always consistent with those evaluated with lower precision.

ref

proof?

Introducing three-valued logic allows Jaguar to decide many expressions that were undecidable before, but is far from a full symbolic evaluation of the expression. For example, in an expression like $p \vee \neg p$, where p is some comparison, should p be inconclusive, the expression will be evaluated as inconclusive, even though logically it must always be true (the well-known tautology/contradiction problem of three-valued logic).

ref

Finally, notice that the three-valued logic is useless if the user re-enables full comparisons (up to "infinite precision"), since the comparison will simply halt instead of returning inconclusive.

Unless all precisions are tried incrementally? Does this make sense?

3.4.3 Lax Comparison

The solution described above for the undecidable comparison problem is as good as it gets: it detects faulty behavior and also allows the user to manage false positives. However, in some situations, the user may wish to disable errors entirely. Take as an example the following program, a simple while loop with an integer counter:

```
1 i := 0;
2 while i < 10 do { i := i + 1; }
```

Listing 3.3: Simple "for" loop with an integer counter variable

The program is meant to exit the loop at 10 iterations, but the comparison $i < 10$ will raise an error when $i = 10$. A simple workaround is to replace the value 10 with a number we know i will never be too close to and that still produces the same results when evaluating the comparison. In this case, 9.5 works. But this puts the chore of finding such a number on the user, and the resulting code is much less readable (and much, much uglier).

But in this particular case, this replacement shouldn't be necessary. Because we know i is restricted

to the integers, we can guarantee that an indeterminate comparison means that $i = 10$ for any precision $n > 1$, since the comparison is only indeterminate when $|i - 10| \leq 2^{-n+1}$. Our restriction of i gives us enough information to make these deductions, effectively bypassing the error.

For these situations, two new operators were created: determinably less than ($<!$) and determinably greater than ($>!$). Unlike the strict comparison operators (the ones introduced in subsection 3.4.1 that can return true, false or indeterminate), these new lax comparators only return true or false: true if the comparison is determinably true, and false in all other cases. They are lax in the sense that they always produce an answer, even when there isn't enough information.

This opens the door to all sorts of misbehavior in the program, of course: $9.99 <! 10$ will evaluate to false if the comparison precision is too low. Indeed, the output of this operator may be incorrect for all comparisons $r <! s$ where $r < s \leq r + 2^{-n+1}$. However, outside this range, it is guaranteed to be correct. So long as this critical range is avoided, the lax comparators can be used safely. If both operands are integers, this works out to $n > 1$, as stated above. For other cases, it may vary. It is the responsibility of the user to make sure the variables being compared are appropriately constrained and choose the appropriate comparison precision for their use case.

3.4.4 Runtime Errors

As seen in the previous sections, our language can throw errors.

semantics of errors?

To handle errors without raising a native Haskell exception, the output of a program can be either a valid state or an error state, containing an error message. The following is a list of all situations that can raise an error:

- An inconclusive boolean expression, caused by a strict comparison (Comparison precision exhausted)
- Too many iterations of all while loops, summed together (Iteration limit exceeded)
- A differential statement with a non-positive or inconclusive time step

3.4.5 Hybrid Undecidability

Another unique problem to the crossing of hybrid systems and arbitrary-precision real numbers is the issue of undecidability on the concatenation point of two hybrid programs. Take the following program:

```
1 x := 0;
```

```
wait 1;
x:=1;
x' = 1 for 1;
```

The output of this program should be a function $x(t)$ such that:

$$\begin{cases} x(t) = 0 & \text{if } 0 \leq t < 1 \\ x(t) = t & \text{if } 1 \leq t \leq 2 \end{cases} \tag{3.1}$$

Although this function is mathematically well-defined, in order to evaluate it to any particular value of t we must perform the comparison $t < 1$ to decide between the two branches of the function. However, because comparison is undecidable on the computable reals, evaluating the function may be a halting computation for $t = 1$, and may take arbitrarily long when evaluated for values close to 1.

To solve this problem, as before, we must use a parametrized comparison, taking in the desired precision of our query. Note that this precision doesn't necessarily need to match the comparison precision used for in-program comparisons. The query precision is only used to choose between two (or more) consecutive hybrid states of the program, leaving the actual execution of the program (variable assignments, choosing between if-else branches and while-loop iterations) completely unaffected. Indeed, the query precision can even be chosen after the whole program has already run, and can be chosen independently for every query.

Taking that into account, to give the user the most flexibility and control, it was decided that a query would be a curried function $\mathbb{R} \to \mathbb{N} \to [\mathbb{R}^n]$ that takes the time t and the desired query precision n and outputs all possible hybrid states within range of that precision. The biggest drawback of this solution is that it may lead to some function branches being evaluated outside their original domain (in this particular example, evaluating the query $t = 0.999$ with a query precision $n < 10$ will result in two possible outputs: 0, the correct output; and 0.999, which is clearly impossible to obtain from the mathematical definition, but must be considered nonetheless).

3.5 Differential Equation Solver

As explained above, the continuous behavior of our hybrid programs is modeled using differential equations. This section exposes how to use a differential equation to evolve a set of initial conditions - the well-known initial value problem.

The general form of the initial value problem (IVP) can be formalized as follows: given the initial value of a smooth unknown function $x : [a, b] \to R^m$ at a such that

indice de abreviat

$$\begin{cases} x(a) = x_0 \\ x'(t) = f(t, x(t)) \end{cases} \tag{3.2}$$

with $f : [a, b] \times CR \times R^m \to R^m$, $m \geq 1$ and C a closed set, find the value of $x(t)$ for any $t \in [a, b]$.

As discussed previously, our solver should work numerically to take advantage of the operations of computable real numbers. In general, numerical methods for solving the initial value problem involve calculating or estimating the first (and/or higher) order derivative(s) of x at a and then estimating $x(a+h)$, for some step size h, with a formula that uses the obtained derivative(s). This logic comes from [... taylor theorem...]. Different methods are then a balancing act between using many derivatives or small step sizes, which are more accurate but more expensive computations, and few derivatives or big step sizes, which are less accurate but much faster to compute, all while preventing compounding errors from all of the estimating. [... exemplos e citacoes]

This puts us in an awkward position: since using any sort of estimation compromises the arbitrary precision we are aiming for, most of these methods are useless to us. The arbitrary precision of computable real numbers requires two things: that the precision of the solution can be increased on demand; and that the solution has a strict error bound. And, ideally, it should be done with the least numerical operations possible, since operations on computable reals are much slower than their floating-point counterparts.

These restrictions naturally lead us to the Taylor method. Formally, the Taylor method can be described as [...TODO...] By using the exact values of the derivatives, the only error introduced is from the truncation of the potentially infinite Taylor series. Acquiring a more accurate approximation is as simple as increasing the order p, which simply means more terms are used, thus reducing the truncation error. Calculating the error bound of the approximation is discussed below.

To implement the Taylor method, we therefore need to calculate the derivatives of x. Although obtaining the first-order derivative is a simple calculation ($x'(a) = f(a, x(a))$), TODO

3.6 Visualizer

3.6.1 Discontinuity Tracking

It was already explained in subsection 3.4.5 how discontinuities (which may arise at the point of concatenation of two hybrid programs) are handled locally, when making individual queries. However, to facilitate a big picture analysis of a program (useful when plotting the system trajectory), Jaguar also includes tools for tracking discontinuity points throughout the whole trajectory. This is as simple as keeping track of every

point at which an assignment is made[1]. Not every assignment generates a discontinuity, since it may not change the state, but checking if the state changed requires comparisons (which, as seen before, leads to problems of comparison precision and undecidability), so all assignments are tracked. The resulting list of (potential) discontinuities is returned alongside the evolution function as the output of a program.

Each discontinuity is located between two differential trajectories. In order to allow matching each discontinuity to its neighboring hybrid branches without directly comparing the time of each (which, again, is undecidable), each discontinuity and differential trajectory also include order information in the form of a step counter. Essentially, alongside the state of the system, Jaguar keeps a counter that starts at zero and increases with both differential statements and assignments. This way, each differential trajectory is associated with a unique step number.

example

A discontinuity is described by all that information put together: a real number corresponding to the time of the discontinuity; the hybrid state and step number "before" the discontinuity (as in, the state the system would have at the moment of discontinuity if the program ended without the assignment statement that caused the discontinuity); and the hybrid state and step number at the moment of the discontinuity.

[1] It is assumed that differential statements do not generate discontinuities in the mathematical sense, since analyzing the differential equation for discontinuities goes beyond the scope of the project

Chapter 4

Results

Chapter 5

Future work

With the goal of implementing a hybrid system simulator in mind, the logical next step of the project is empirically testing the surveyed packages in a benchmarking setting, focusing on correctness but also taking performance into account, to decide on the most appropriate for use. This process will necessarily go through the creation of a standard test suite of exact real computations, and the evaluations of the packages can be compared to the predictions of section 2.2. This process is expected to take about three weeks.

After the selection of the library, the implementation of the hybrid simulator may begin. This will take the implementation of a parser, a numeric differential equation solver and an interpreter. Of these, the derivative equation solver is the most sensitive component, and will require further research to design and fine-tune. Taking this into account, the whole implementation should take between five and six weeks.

This simulator can then be tested against other existing hybrid simulators such as Lince to demonstrate their differences in performance and predictive power. This benchmarking should mostly focus on accuracy and performance tests, and the trade-off between the two, though other metrics, such as memory consumption, are also relevant. The implemented simulator is expected to have a worse performance when simulating with the same precision as Lince, but be able to simulate arbitrarily precise approximations of the answer. This evaluation is expected to take two to three weeks.

All these steps will be documented on a final dissertation presenting the faced challenges, engineered solutions and collected results. The writing of this dissertation will likely take over a month, and is the last step of this project.